
Automotive SoC Motor Driver

1.0 DIRECT MEMORY ACCESS (DMA)

1.1 Features

DMA supports the following features

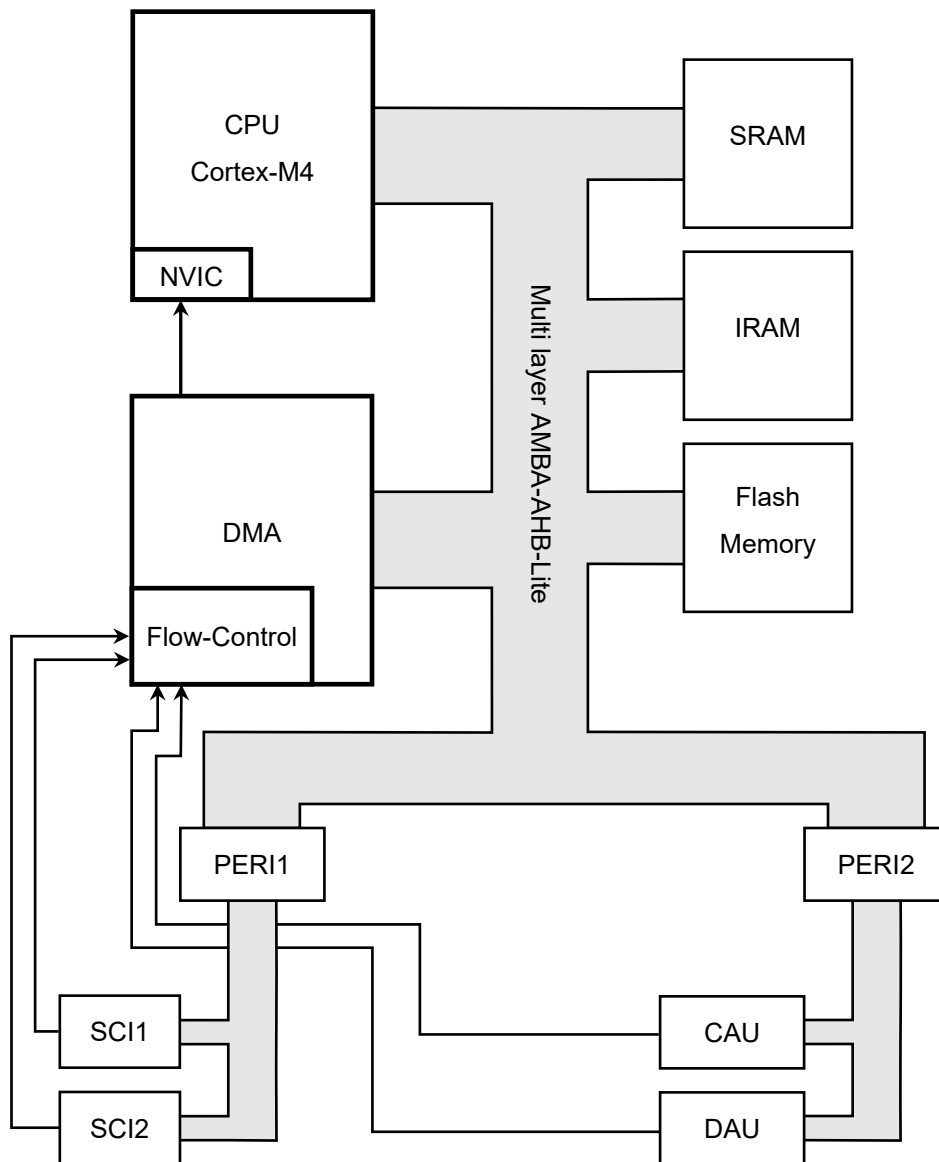
- Memory-based, linked-list transfer descriptors
 - 3 Descriptor sizes to balance functionality and setup overhead
- 3 DMA channel
- Supports Multiple data transfer
 - Memory to Memory Transfer
 - Peripheral to Memory Transfer
 - Memory to Peripheral Transfer
 - FIFO to Memory Transfer
 - Memory to FIFO Transfer
- Programmable X and Y count, increment, access size and burst length
- AMBA AHB-Lite master interface for simultaneous read and write
- Scatter and gather functionality

1.2 Introduction

The A89201 features 3 DMA channels. Each channel is independent and configurable separately. The data transfer is fully optimized and generally provides higher data bandwidth comparing with CPU driven data transfer. The DMA fully compatible with AMBA AHB lite interface. The AMBA bus has multilayer structure so DMA access is optimized please refer CPU section for more detail. The DMA supports multiple ways to trigger for initiating transfer and multiple type of memory structure, such as FIFO. The DMA can perform scatter and gather function without CPU intervention.

The DMA has access to Flash Memory, SRAM, IRAM, Peripherals. A DMA operation can also be triggered by one of the following peripherals: SCI0, SCI1, CAU, and DAU.

2.0 BLOCK DIAGRAM



3.0 REGISTER SET

Each DMA channel has dedicated register sets to control DMA behavior.

Base Address: 0x40070000 (DMA1), 0x40070080 (DMA2), 0x40070100 (DMA3)

Table 1: Register map of the DMA

Register Name	Access Type	Address offset	Reset Value
status	R/W	0x40	0x0003
control	R/W	0x44	0x0000
current	R/W	0x48	0x0000

3.1.1 Register Definition

Status Register

Offset Address = 0x40
Reset Value = 0x0003

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	-	ER	DCO	DC	AC
U	U	U	U	R/1C	R/1C	R/1C	Note1

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0
Res: Reserved bit

Field	Bits	Description
ER	3	Indicates an error occurred during (such as hresp not OK during a read or write or an unsupported value in a descriptor). 0: No error 1: Error
DCO	2	Descriptor complete overflow. This indicates the DC flag was already set when a descriptor was completed. 0: No overflow 1: Overflow
DC	1	Descriptor complete 0: Not complete 1: Complete
AC	0	All descriptors complete 0: Not Complete 1: Complete

Note1: to clear AC bit, below procedure must be followed:

1. Disable the DMA.
2. Clear the DC bit by R1C.
3. Change the current register value to have non-zero (dummy) value to clear the AC bit.
4. Changing the current register without enabling DMA will clear AC and won't initiate another DMA operation

To perform a new operation, update the current address with correct descriptor address then enable the DMA to initiate execution.

Control Register

Offset Address = 0x44

Reset Value = 0x0000

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	-	-	SP	ERIE	DCIE	-	E
U	U	U	R/W	R/W	R/W	R/W	R/W

Legend R: Read W: Write 1C: Write 1 to clear	U: Unimplemented bit, read as 0 Res: Reserved bit
--	--

Field	Bits	Description
SP	4	Stop transfer immediately. Data may be lost. 0: Continue 1: Stop
ERIE	3	Error interrupt enable 0: Disabled 1: Enabled
DCIR	2	Descriptor complete interrupt enable. 0: Disabled 1: Enabled
-	1	This bit field shall be set as 0.
E	0	Enable DMA 0: Disabled 1: Enabled

Current Register

Offset Address = 0x48
Reset Value = 0x0000

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
current<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
current<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
current<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
current<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
current<31:0> ⁽¹⁾	31:0	The current address register contains the address of the current descriptor being processed. When the DMA is idle, the current register should be written with the address of the first descriptor in order to start processing. Writing the current register with 0, will set <i>status.AC</i> to 1.

4.0 DESCRIPTOR

The descriptor can be allocated in any readable address space. For instance, it can be in SRAM, IRAM, Flash. Once the first address of descriptor is passed to the current register, the DMA will read descriptor and process it as in the descriptor.

The tiny descriptor for basic transfers that minimizes memory footprint and descriptor read overhead.

The small descriptor that supports the same functionality as the tiny descriptors with the addition of the next_address field for changing descriptors.

The full descriptor that has a larger memory footprint but offers full use of all the DMA features.

The small and full descriptor can be chained together via the next_address field, when the commanded descriptor operation is completed, the DMA will process a descriptor which is specified in next_address field.

4.1.1 Tiny Descriptor

Table 2: Register map of the Tiny Descriptor

Register Name	Data size	Address offset
desc_control	32bit	0x00
src_address	32bit	0x04
dat_address	32bit	0x08
count	32bit	0x0C

4.1.2 Small Descriptor

Table 3: Register map of the Small Descriptor

Register Name	Data size	Address offset
desc_control	32bit	0x00
next_address	32bit	0x04
src_address	32bit	0x08
dat_address	32bit	0x0C
count	32bit	0x10

4.1.3 Full Descriptor

Table 4: Register map of the Full Descriptor

Register Name	Data size	Address offset
desc_control	32bit	0x00
next_address	32bit	0x04
src_address	32bit	0x08
dst_address	32bit	0x0C
src_control		0x10
dst_control	32bit	0x14
count_x	32bit	0x18
count_y	32bit	0x1C
src_inc_x	32bit	0x20
src_inc_y	32bit	0x24
dst_inc_x	32bit	0x28
dst_inc_y	32bit	0x2C

desc_control (Descriptor)

Offset Address = 0x00 (Tiny and Small)

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
PIDX<5:0>						BURST<1:0>	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
SIZE<1:0>		MODE<2:0>			DCIE	DT<1:0>	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
PIDX<5:0> ⁽¹⁾	15:10	Peripheral index. Determines which peripheral to use for flow control, if MODE is not 0. 00000: SC10 00010: CAU Other Selections shall not be chosen 00001: SC11 00011: DAU
BURST<1:0>	9:8	Length of Burst 00: 1 beat 01: 4 beats 10: 8 beats 11: 16 beats
SIZE<1:0>	8:7	Size of each access. 00: 1 byte 01: 2 bytes 10: 4 bytes 11: Shall not be chosen
MODE<2:0>	5:3	Transfer Mode 000: Memory to Memory 001: Memory to Peripheral 010: Peripheral to Memory 011: Memory to FIFO 100: FIFO to Memory 101: None to Memory 110: Memory to None 111: Shall not be chosen
DCIE	2	Descriptor complete interrupt enable 0: Disable Interrupt 1: Enable Interrupt
DT<1:0>	1:0	Descriptor Type 00: Shall not be chosen 01: Small 10: Tiny 11: Shall not be chosen

desc_control (Descriptor)

Offset Address = 0x00 (Full)

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
-	-	-	-	-	-	-	-
W0	W0	W0	W0	W0	W0	W0	W0

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
					DCIE	DT<1:0>	
W0	W0	W0	W0	W0	R/W	R/W	R/W

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

W0: Shall write as 0

Res: Reserved bit

Field	Bits	Description
DCIE	2	Descriptor complete interrupt enable 0: Disable Interrupt 1: Enable Interrupt
DT<1:0>	1:0	Descriptor Type 00: Full 01: Shall not be chosen 10: Shall not be chosen 11: Shall not be chosen

Next Address (Descriptor)

Offset Address = 0x08 (Small)
0x04 (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
next_address<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
next_address<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
next_address<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
next_address<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
next_address<31:0> ⁽¹⁾	31:0	32bit address for next descriptor

Source Address (Descriptor)

Offset Address = 0x04 (Tiny)
0x08 (Small, Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
src_address<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
src_address<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
src_address<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
src_address<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
src_address<31:0> ⁽¹⁾	31:0	Transfer source address 32bit address

Destination Address (Descriptor)

Offset Address = 0x08 (Tiny)
0x0C (Small, Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
dst_address<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
dst_address<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
dst_address<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
dst_address<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
dst_address<31:0> ⁽¹⁾	31:0	Transfer destination 32bit address

Count (Descriptor)

Offset Address = 0x0C (Tiny)

Offset Address = 0x10 (Small)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
cout<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
cout<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
cout<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
cout<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
count<31:0> ⁽¹⁾	31:0	32bit Unsigned integer for count

Source Control (Descriptor)

Offset Address = 0x10 (Full)

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
PIDX<5:0>						-	-
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	BURST<2:0>			SIZE<1:0>		T<1:0>	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
PIDX<5:0> ⁽¹⁾	15:10	Peripheral index to determine the source of flow control. 00000: SC10 00010: CAU Other Selections shall not be chosen 00001: SC11 00011: DAU
BURST<2:0>	6:4	Maximum length of burst. This drives the AHB <i>hburst</i> signal. The actual burst length may be shorter, depending upon the FIFO size and count of elements to transfer. 000: Undefined (INCR) 010: 4 beats (INCR4) 100: 16 beats (INCR16) 001: 1 beat (SINGLE) 011: 8 beats (INCR8) 101~111: Shall not be chosen
SIZE<1:0>	3:2	Size of each access when T=1. This drives the AHB <i>hsize</i> signal. <i>src_address</i> must be aligned according to the transfer size. When T=0, this should be set to 0, and access size will be determined automatically depending upon count and alignment. 00: 1 byte 10: 4 bytes 01: 2 bytes 11: Shall not be chosen
T<1:0>	1:0	Source address type. When set to peripheral type, the <i>rx_ready[PIDX]</i> signal from the peripheral must be asserted before a transfer will take place. When set to memory, the transfer will occur unconditionally. When set to non, <i>src_address</i> is not used and source data will be just zeros. 00: Memory 10: None 01: Peripheral 11: Shall not be chosen

Destination Control Descriptor

Offset Address = 0x14 (Full)

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
PIDX<5:0>						-	-
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
-	BURST<2:0>			SIZE<1:0>		T<1:0>	
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
PIDX<5:0> ⁽¹⁾	15:10	Peripheral index to determine the source of flow control. 00000: SC10 00001: SC11 00010: CAU 00011: DAU Other Selections shall not be chosen
BURST<2:0>	6:4	Maximum length of burst. This drives the AHB <i>hburst</i> signal. The actual burst length may be shorter, depending upon the FIFO size and count of elements to transfer. 000: Undefined (INCR) 001: 1 beat (SINGLE) 010: 4 beats (INCR4) 011: 8 beats (INCR8) 100: 16 beats (INCR16) 101~111: Shall not be chosen
SIZE<1:0>	3:2	Size of each access when T=1. This drives the AHB <i>hsize</i> signal. <i>dst_address</i> must be aligned according to the transfer size. When T=0, this should be set to 0, and access size will be determined automatically depending upon count and alignment. 00: 1 byte 01: 2 bytes 10: 4 bytes 11: Shall not be chosen
T<1:0>	1:0	Source address type. When set to peripheral type, the <i>rx_ready[PIDX]</i> signal from the peripheral must be asserted before a transfer will take place. When set to memory, the transfer will occur unconditionally. When set to non, <i>dst_address</i> is not used and source data will be discarded. 00: Memory 01: Peripheral 10: None 11: Shall not be chosen

Count X Descriptor

Offset Address = 0x18 (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
count_x<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
count_x<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
count_x<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
count_x<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
count_x<31:0> ⁽¹⁾	31:0	The count X register contains the number of transfer that should occur in the X dimension. The number of bytes transferred in the X dimension is $(1 \ll \text{src_control.SIZE}) * \text{count_x}$. If src_control.SIZE and dst_control.SIZE differ, the count of transfers to the destination, is $\text{count_x} * (1 \ll \text{src_control.SIZE}) / (1 \ll \text{dst_control.SIZE})$. For transfers where the address type is memory (src_control.T or $\text{dst_control.T} = 0$), reads or writes can be coalesced, providing that the address is appropriately aligned, count is sufficiently high and the address increment is the same as the size. For example, specifying a copy where $\text{control.T}=0$, $\text{control.SIZE}=0$, $\text{address}=0$, $\text{count_x}=4$, $\text{inc_x}=1$ can result in a single word accesses, instead of 4 byte accesses. Coalescing is not performed when the address type is peripheral.

Count Y Descriptor

Offset Address = 0x1C (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
count_y<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
count_y<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
count_y<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
count_y<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
count_y<31:0> ⁽¹⁾	31:0	The count Y register contains the number of iterations, minus 1, of transfers in the X direction (the Y dimension). So, for a 1 dimensional transfer, set count_y to 0. The total number of bytes transferred is $((1 \ll \text{src_control.SIZE}) * \text{count_x}) * (\text{count_y} + 1)$.

Source Increment X Descriptor

Offset Address = 0x20 (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
src_inc_x<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
src_inc_x<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
src_inc_x<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
src_inc_x<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
src_inc_x <31:0> ⁽¹⁾	31:0	The source increment X register contains a signed integer that is added to the src_address register after each transfer in the X dimension.

Source Increment Y Descriptor

Offset Address = 0x24 (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
src_inc_y<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
src_inc_y<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
src_inc_y<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
src_inc_y<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
src_inc_y<31:0> ⁽¹⁾	31:0	The source increment Y register contains a signed integer that is added to the src_address register after each transfer in the Y dimension.

Destination Increment X Descriptor

Offset Address = 0x28 (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
dst_inc_x<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
dst_inc_x<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
dst_inc_x<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
dst_inc_x<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
dst_inc_x<31:0> ⁽¹⁾	31:0	The destination increment X register contains a signed integer that is added to the <i>dst_address</i> register after each transfer in the X dimension.

Destination Increment Y Descriptor

Offset Address = 0x2C (Full)

bit31	bit30	bit29	bit28	bit27	bit26	bit25	bit24
dst_inc_y<31:24>							
R/W							

bit23	bit22	bit21	bit20	bit19	bit18	bit17	bit16
dst_inc_y<23:16>							
R/W							

bit15	bit14	bit13	bit12	bit11	bit10	bit9	bit8
dst_inc_y<15:8>							
R/W							

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
dst_inc_y<7:0>							
R/W							

Legend

R: Read

W: Write

1C: Write 1 to clear

U: Unimplemented bit, read as 0

Res: Reserved bit

Field	Bits	Description
dst_inc_y<31:0> ⁽¹⁾	31:0	The destination increment Y register contains a signed integer that is added to the <i>dst_address</i> register after each transfer in the Y dimension.

5.0 INTERRUPT

The DMA supports the following interrupt.

- Per-channel all descriptors complete interrupt
- Per-channel descriptor complete interrupt
- Per-channel error interrupt

The descriptors complete interrupt will be raised when the DMA has finished the transfer specified by the current descriptor. The DC flag in the status register will be set 1 to indicate this. When the DC flag in the status register is set to 1 and the DCIE flag in the control register is set to 1, the descriptors complete interrupt will be asserted.

The error interrupt will be raised then the ER flag in the status register is 1 and the ERIE flag in the control register is set to 1. This indicates an error was detected during either the reading of a descriptor or the transfer specified by the descriptor.

6.0 ERROR HANDLING

When the DMA detects a read or write error on the AHB bus, either the reading of the descriptor will be terminated, or the transfer will be terminated. The termination may not be immediate, as any burst in progress will be allowed to complete.

The DMA will set the ER flag in the status register to indicate the error, and no further transfers will take place on the corresponding channel until the flag is cleared. Before clearing the flag, the current register should be set to NULL, otherwise the descriptor will be re-read. If chained descriptors are being used, the current register should contain the address of the descriptor that caused the error.

7.0 STOPPING TRANSFER

A transfer on DMA channel can be stopped by writing to the channel's control.SP register. The transfer may not stop immediately, as a burst that is in progress may need to be completed. When the transfer is stopped, the control.E register will be cleared to indicate this.

To start a new transfer, the current register should first be written with 0, to set the status.AC flag. The channel should be reenabled by setting control.E to 1, and then the address of the new descriptor can be written to the current register.

8.0 CHANNEL PRIORITY

Transfers on the lowest valid channel have priority. For example, if the transfer specified by channel 0 is ready to be processed at the same time as the transfer on channel 1, channel 0 will be completed first. A higher channel (with lower priority) may be switched to part way through the transfer on a lower channel (with higher priority); if the transfer on the lower channel cannot be completed due to a peripheral indicating it is not ready.

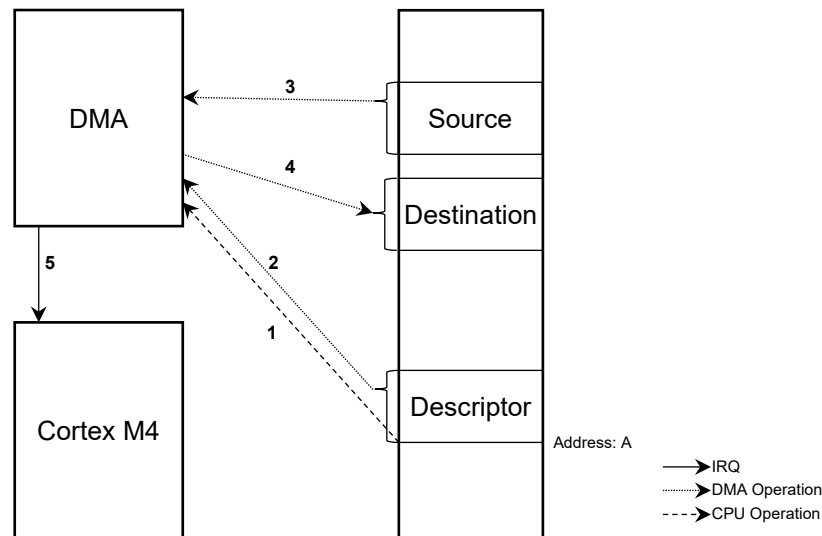
9.0 APPLICATION INFORMATION

In this section describe typical use case of the DMA.

9.1 Operation of descriptor

9.1.1 DMA operation

Figure1: Descriptor Operation (Memory to Memory)

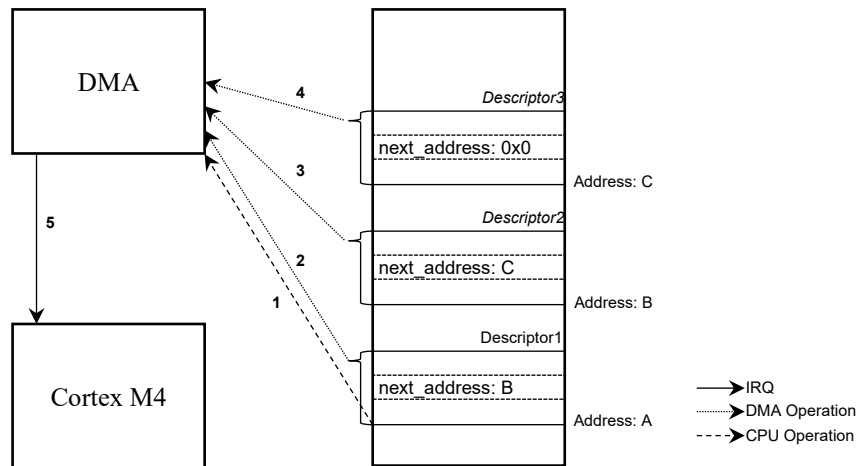


The descriptor operation summary is described in Figure 1. To command DMA transfer, the descriptor shall be prepared in a read memory region, such as SRAM, IRAM, Flash. Once descriptor is prepared, CPU need to transfer first address, address A in the figure of descriptor to the current register in DMA to trigger its transfer. This operation is indicated as 1 in Figure1.

Once DMA receive non 0 address in the current register, DMA will read entire descriptor and configure the operation as specified in the descriptor. This example shows a memory to memory transfer. When DMA is ready, DMA read source region and write into destination address as shown as 3 and 4 in Figure1. When specified operation is completed, DMA will generate IRQ request, if descriptor complete enable bit, ERIE on control register set 1.

9.1.2 Descriptor chain

Figure 2: Descriptor Chain



The DMA can chain the operation. This function can be used as scatter and gather operation.

Figure 2 shows an example of three descriptor chaining. First of all, multiple descriptors should be prepared for the chaining operation. The Tiny descriptor does not support descriptor chaining, so it shall be Small or Full descriptor.

Once necessary descriptors are prepared, first address of descriptor1 need to be send out to the current register. This operation is indicated as 1 in Figure 2. Once DMA received first descriptor address, then DMA process descriptor 1 and complete it. Once the transfer is completed, DMA read next descriptor which is specified on `next_address` in the descriptor 1. Once DMA is ready to transfer with descriptor 2, DMA automatically start transfer as commanded in descriptor2 and complete it. The descriptor3 is same operation as first chain. If the `next_address` in a descriptor specified as null (all 0), the DMA chain will be terminated and generate IRQ flag to CPU and it is indicated as DC and AC in the status register.

9.2 Use case

The DMA supports multiple way to configure it. In this section typical examples of descriptor for each senario are shown.

9.2.1 Memory Set

Memory set instruction can be achieved with DMA assistance. For example, set same data to multiple sequential memory space is possible. This section shows reference configuration for descriptor.

Descriptor Example: 1

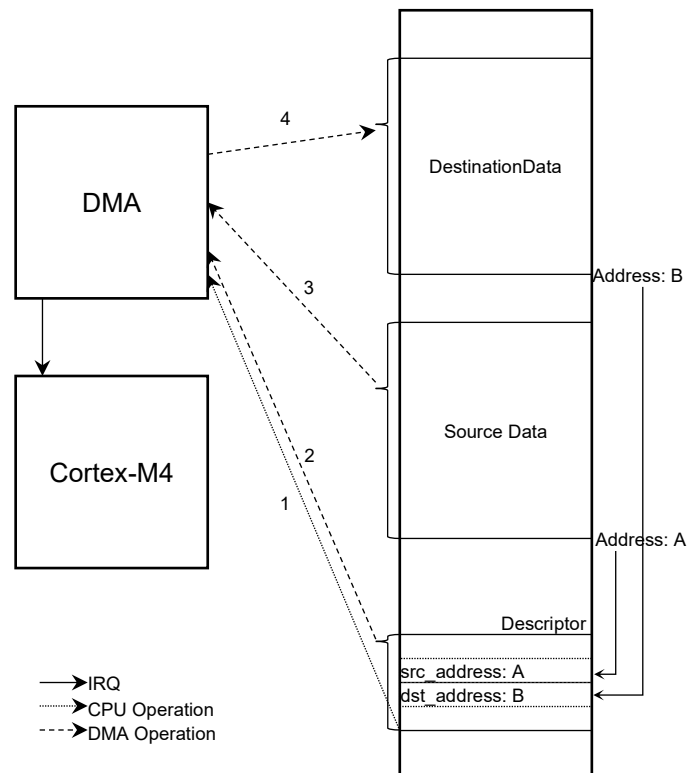
```

dma.desc_control: Full descriptor
dma.next_address: NULL
dma.src_address: Pointer of data to initialize
dma.dst_address: First data address to initialize
dma.src_control: T: Memory, Size: Any, Burst: Any, PIDX: Does not matter
dma.dst_control: T: Memory, Size: Any, Burst: Any, PIDX: Does not matter
dma.count_x: Required size in unit of SIZE on control setting
dma.count_y: 0
dma.src_inc_x 0
dma.src_inc_y 0
dma.dst_inc_x 1
dma.dst_inc_y 0

```

The full desciotr shall be chosen for Memory set operation. The data, which will be copied shall be prepared in an address and give the address to src_address. The transaction size can be any, which can be specified in src_control and dst_control but it shall be consistent.

Figure 3: Operation block diagram



When the CPU transfer the first address of descriptor, DMA start operation, shown as 1 on Figure 3. When the DMA accept the current address, then DMA read descriptor, shown as 2 on Figure 3. As the DMA commanded by descriptor data will be transferred, shown as 3 and 4 on Figure 3.

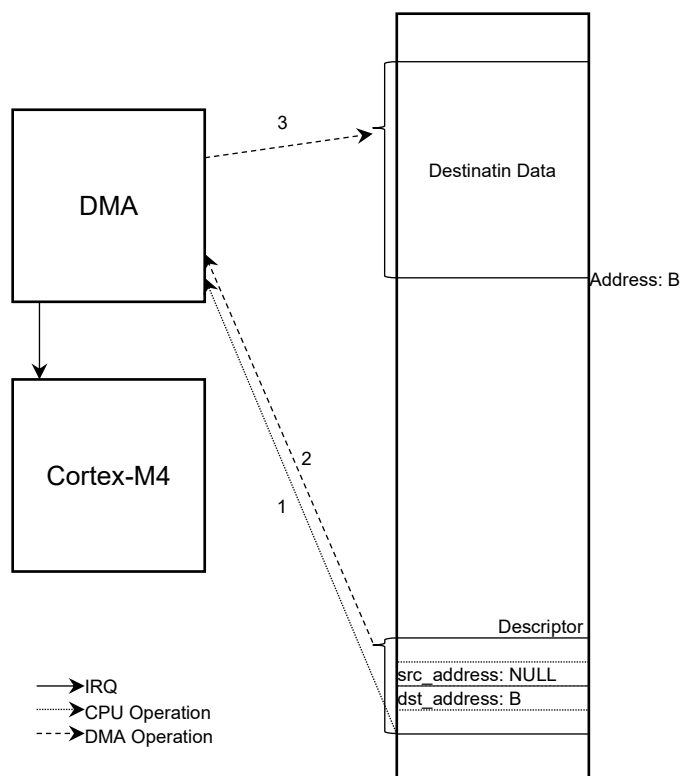
9.2.2 Memory Zero

The DMA can be used to initialize memory to zero. For example, if any sequential address memory area need to set 0, this example can be suite.

```
dma.desc_control: DT: Tiny, DCIE: Any, MODE: None to Memory, SIZE: Any, BURST: Any, PIDX: Any
dma.src_address:  NULL (All zero)
dma.dst_address:  First data address to initialize
dma.count:       Required size in unit of SIZE on the control setting
```

The descriptor can be Tiny. The operation example code is for Tiny descriptor. If the initializing area is scattered, the Small descriptor can be chosen and those can be chained using `next_address`. MODE shall be none to memory type. If it is selected, source address is ignored and set the original data as all zero.

Fig4: Operation block diagram



When the DMA is in operation with the non to memory mode, the source address is ignored and write all 0 to destination address.

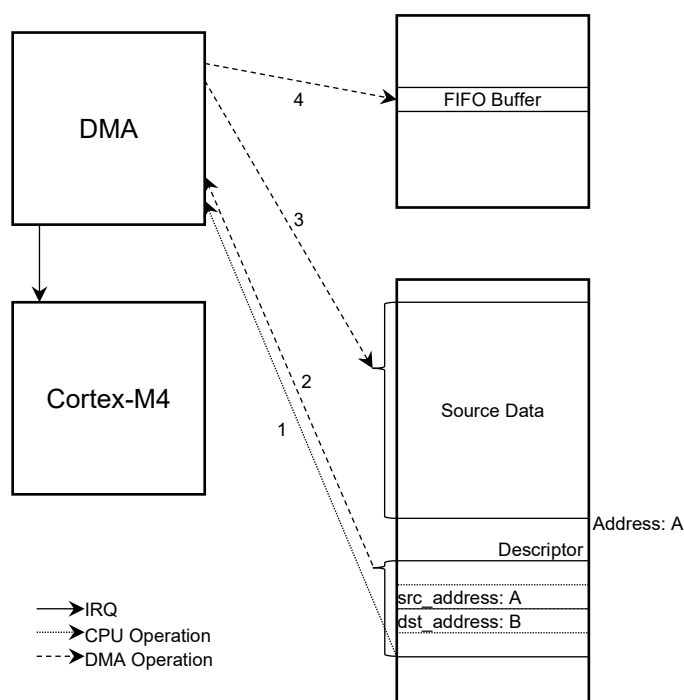
9.2.3 Memory to FIFO

The memory to FIFO can be used such like from SRAM to FIFO on SCI. When a SW prepared the data, it can be called to send out by DMA.

```
dma.desc_control: DT: Tiny, DCIE: Any, MODE: Memory to FIFO, SIZE: Any, BURST: Any, PIDX: Any
dma.src_address:  First address of data
dma.dst_address:  Address of FIFO buffer
dma.count:        Required size in unit of SIZE on the control setting
```

The example uses Tiny descriptor. In this example data array which is aligned in sequential wil be transferred to signle FIFO address when it is triggered. The count should be matched with data size.

Fig5: Operation Diagram



When the descriptor pass to DMA (1 on Fig 5), the DMA read the descriptor and pass the source data sequentially to FIFO buffer (3 and 4 on Fig 5), which is specified as address B.

9.2.4 Double Buffering (Ping-Pong Buffers)

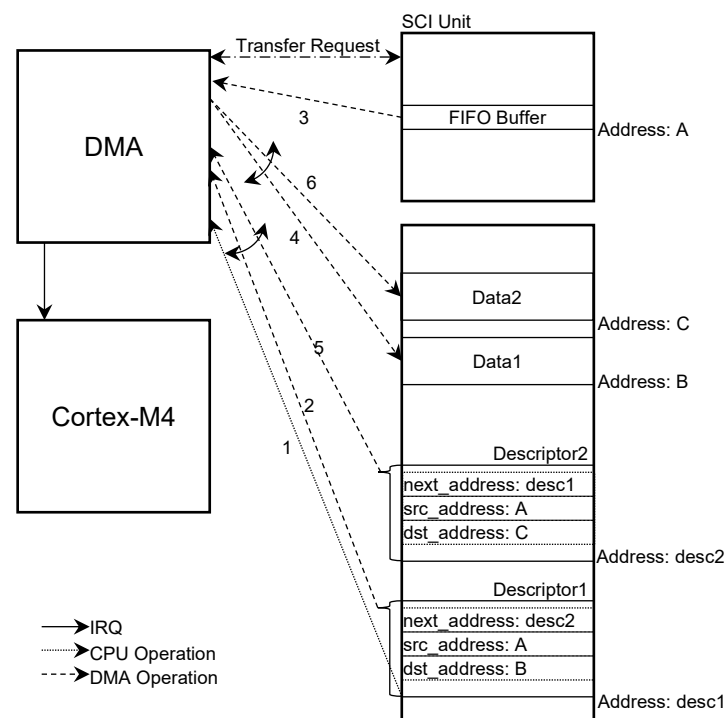
A ping pong buffer can be implemented using DMA. If the CPU resource is occupied for long period, peripheral buffer might be overwritten. A ping pong buffer can be used to avoid data loss. Two descriptor is chained each other, and both requests transfer the data to different buffers.

```
[Descriptor1]
dma.desc_control: DT: Tiny, DCIE: Any, MODE: FIFO to Memory, SIZE: Any, BURST: Any, PIDX: Any
dma.next_address: First address of the descriptor 2
dma.src_address: First address of data1
dma.dst_address: Address of FIFO buffer
dma.count: Required size in unit of SIZE on the control setting

[Descriptor2]
dma.desc_control: DT: Tiny, DCIE: Any, MODE: FIFO to Memory, SIZE: Any, BURST: Any, PIDX: Any
dma.next_address: First address of the descriptor 1
dma.src_address: First address of data2
dma.dst_address: Address of FIFO buffer
dma.count: Required size in unit of SIZE on the control setting
```

When the CPU enables the DMA, descriptor 1 will wait for the trigger from the peripheral. When trigger is generated, DMA transfer the data to data1 storage and descriptor will change to descriptor2. So, DMA will transfer second data to data2 storage. When each IRQ is inserted, CPU can take appropriate action and swap the buffer.

Figure6: Operation Diagram



When the descriptor 1 is accepted (shown 1 and 2 in Figure 6), the DMA will wait the DMA request from SCI unit. When SCI send the request, the data will be transmitted (shown 3 in Figure6). The data will be transmitted to data1 as commanded in descriptor1. When the first transmission is completed, the descriptor is change to descriptor2 and wait second transmission request from SCI unit. When it request the transfer, the data will be transmitted to the data2. When it is completed, the descriptor change back to descriptor1. Each time transmission completes, DMA request the interrupt to the CPU.

Revision History:

Date	Contents changed
March 3 2022	Removed bus registers which is not accessible from the AMBA bus.
March 25 2022	Updated the register map.
April 26 2022	Updated the register map
July 10 2025	Added AC bit in status register Removed LIN selection for DMA operation
Feb 20 2026	Corrected status register default value

Copyright ©2026, Allegro MicroSystems, Inc

Allegro MicroSystems, Inc reserves the right to make, from time to time, such departures from the detail specifications as may be required to permit improvements in the performance, reliability, or manufacturability of its products. Before placing an order, the user is cautioned to verify that the information being relied upon is current.

Allegro's products are not to be used in life support devices or systems, if a failure of an Allegro product can reasonably be expected to cause the failure of that life support device or system, or to affect the safety or effectiveness of that device or system.

The information included herein is believed to be accurate and reliable. However, Allegro MicroSystems, Inc assumes no responsibility for its use; nor for any infringement of patents or other rights of third parties which may result from its use.